

GDB debugging workshop for Mobileye

Alex Kushnir, Amir Kirsh, Adam Segoli Schubert

© All rights reserved.
Materials are for the sole use of the course participants, October 2026. Any other use is forbidden

About Claritune



Tailored Technical Courses:

- Modern C++ for Embedded Developers
- Secure Design and Development in C++
- Effective Unit Testing with GoogleTest
- C++ Multithreading and Concurrency
- Design Patterns for C++ Developers
- Linux Fundamentals
- DevOps: Docker
- DevOps: Kubernetes

and more ...

CTO: Amir Kirsh

Previously the Chief Programmer at Comverse.

Lecturer at the Academic College of Tel-Aviv-Yaffo and Tel-Aviv University.

A team of top-level experts.

Trusted by a wide range of leading high-tech companies.



GDB debugging workshop

Session 1

Introduction



Workshop agenda

Session 1

GDB basics

GDB TUI

Debug symbols

GDB in IDE

Advanced breakpoints

Session 2

Coredump analysis

Attaching to a process

Altering execution

Tracepoints

Checkpoints

Reverse execution

Remote debugging

Memory issues

Session 3

Multithreading

GDB customizations

Complementary tools

What's new in C++ standard?

From: “It crashed” to: “I know exactly why”

Well... debugging is not only for fixing crashes

Debugging is understanding program state, exact flow, wrong assumptions, race conditions and logical bugs.



The first rule of Debugging

... ?


THE FIRST RULE OF
DEBUGGING
IS _____.

The first rule of Debugging is _____

Debugging must be methodical!

A methodical debugging flow

1. **Ask the question:** Why is X == 7 in flow Y?
2. **State the expectation:** X should be 9, because…
3. **Form a model:** What do I think is happening?
4. **Step through the flow:** Check each step against your model.

Methodical debugging approaches

1. **Divide and conquer:** Reproduce the bug in a smaller, simpler scenario. If it still happens, that's useful. If it doesn't, that's useful too.
2. **Record each run (open a doc for it!):** Note the scenario, whether the bug appeared, and any relevant observations.
3. **Change one thing at a time:** Avoid multiple edits in one iteration, so each run gives a clear signal. Document the change you made.

A methodical bug fix flow (1)

1. Do not fix a bug until you have a **solid theory** for its **root cause**.
2. If you have a theory about the root cause but you are not sure, **prove it first**, with a minimal external example.
3. Before pushing the fix, **add a unit test** that fails on the bug and passes with the fix.

A methodical bug fix flow (2)

4. **Keep commits isolated**; do not mix unrelated changes with the fix, (relevant logging if needed can be considered part of the fix).
5. **Require code review before merging**: explain the bug, the root cause, and why the fix is correct.

Tools for debugging

- Logs and printouts
- Statistics
- Debugger
- Other tools (valgrind, procdump, Wireshark, etc.)

Use GDB!

- for ***analyzing program flow*** – if you can't see the bug in a quick code review or in the logs, start a debugging session!

- for ***analyzing crashes and hangs***

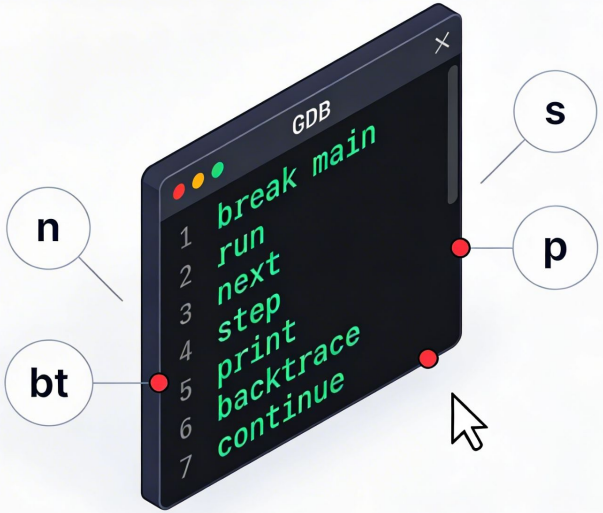
Debugging with GDB

Build with ***debug information***, reproduce the relevant flow, stop at the right places with ***breakpoints*** *or* ***watchpoints***, then ***inspect*** program state:

- stack frames
- variables, memory, registers
- ... and the exact point where behavior diverges from expectations.

GDB basics

Command line for the win!



Basic GDB usage demo

`day01/00_basic_example`

Let’s start the GDB session by typing

```
gdb build/hello
```

Basic GDB commands

- **`list`** - displays source code lines around a specific location
- **`run`** - Run the loaded program
- **`start`** - Run the loaded program and stop at main
- **`continue`** - Resume program execution after a breakpoint
- **`next`** - Step-over
- **`step`** - Step-into
- **`finish`** - Step-out
- **`backtrace`** - Show call stack
- **`quit`** - Exit GDB

Basic GDB usage demo - explained

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from build/hello...
(gdb) b main
Breakpoint 1 at 0x4811: file /home/alex/gdb_workshop/test/hello.cpp, line 5.
(gdb) run
Starting program: /home/alex/gdb_workshop/test/build/hello
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at /home/alex/gdb_workshop/test/hello.cpp:5
5               printHello();
(gdb) step
printHello () at /home/alex/gdb_workshop/test/printer.cpp:6
6               std::println("Hello, world!");
(gdb) finish
Run till exit from #0  printHello () at /home/alex/gdb_workshop/test/printer.cpp:6
Hello, world!
Demonstration of finish command
main () at /home/alex/gdb_workshop/test/hello.cpp:6
6               return 0;
(gdb) cont
Continuing.
[Inferior 1 (process 26251) exited normally]
(gdb)
```

Set a breakpoint in main()

Basic GDB usage demo - explained

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from build/hello...
(gdb) b main
Breakpoint 1 at 0x4811: file /home/alex/gdb_workshop/test/hello.cpp, line 5.
(gdb) run
Starting program: /home/alex/gdb_workshop/test/build/hello
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at /home/alex/gdb_workshop/test/hello.cpp:5
5               printHello();
(gdb) step
printHello () at /home/alex/gdb_workshop/test/printer.cpp:6
6               std::println("Hello, world!");
(gdb) finish
Run till exit from #0  printHello () at /home/alex/gdb_workshop/test/printer.cpp:6
Hello, world!
Demonstration of finish command
main () at /home/alex/gdb_workshop/test/hello.cpp:6
6               return 0;
(gdb) cont
Continuing.
[Inferior 1 (process 26251) exited normally]
(gdb)
```

Set a breakpoint in main()

Run the program

Basic GDB usage demo - explained

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from build/hello...
(gdb) b main
Breakpoint 1 at 0x4811: file /home/alex/gdb_workshop/test/hello.cpp, line 5.
(gdb) run
Starting program: /home/alex/gdb_workshop/test/build/hello
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

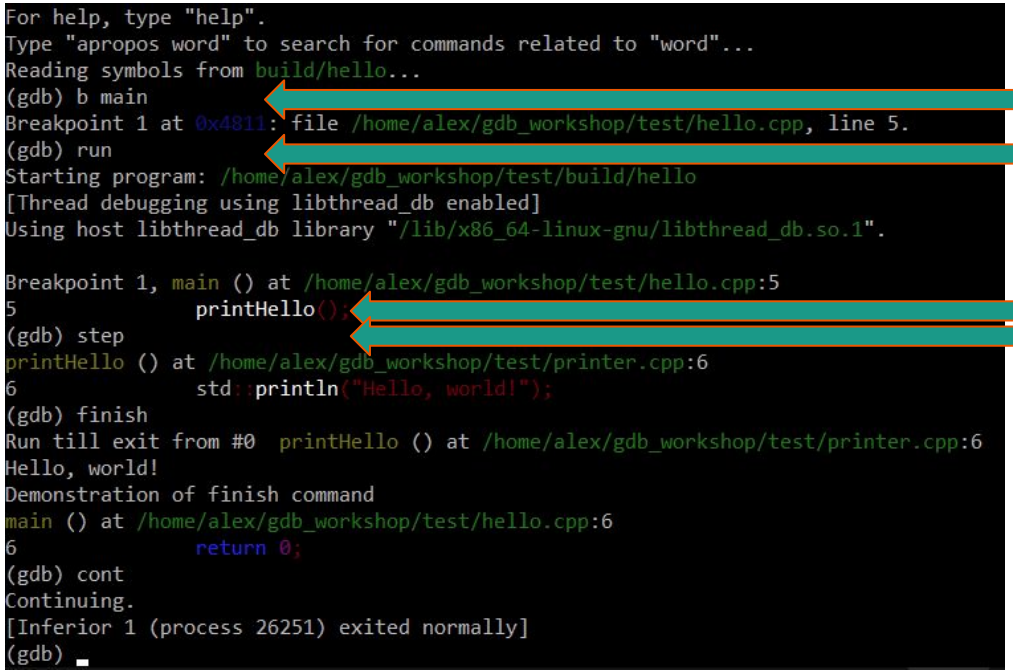
Breakpoint 1, main () at /home/alex/gdb_workshop/test/hello.cpp:5
5               printHello();
(gdb) step
printHello () at /home/alex/gdb_workshop/test/printer.cpp:6
6               std::println("Hello, world!");
(gdb) finish
Run till exit from #0  printHello () at /home/alex/gdb_workshop/test/printer.cpp:6
Hello, world!
Demonstration of finish command
main () at /home/alex/gdb_workshop/test/hello.cpp:6
6               return 0;
(gdb) cont
Continuing.
[Inferior 1 (process 26251) exited normally]
(gdb)
```

Set a breakpoint in main()

Run the program

Breakpoint hit

Basic GDB usage demo - explained



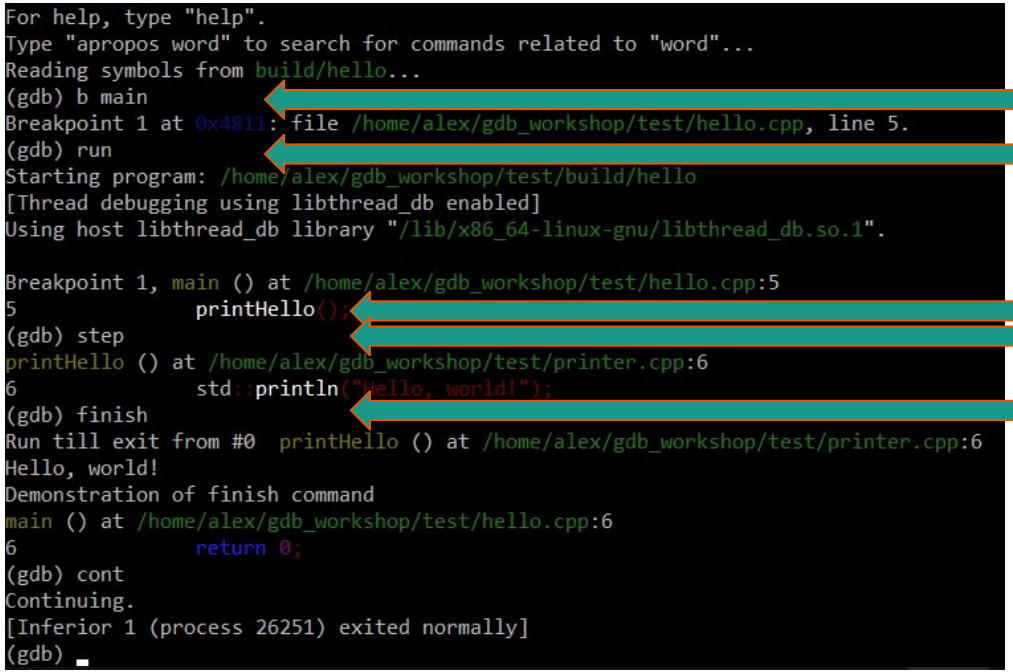
Set a breakpoint in main()

Run the program

Breakpoint hit

Step into the printHello() function

Basic GDB usage demo - explained



Set a breakpoint in main()

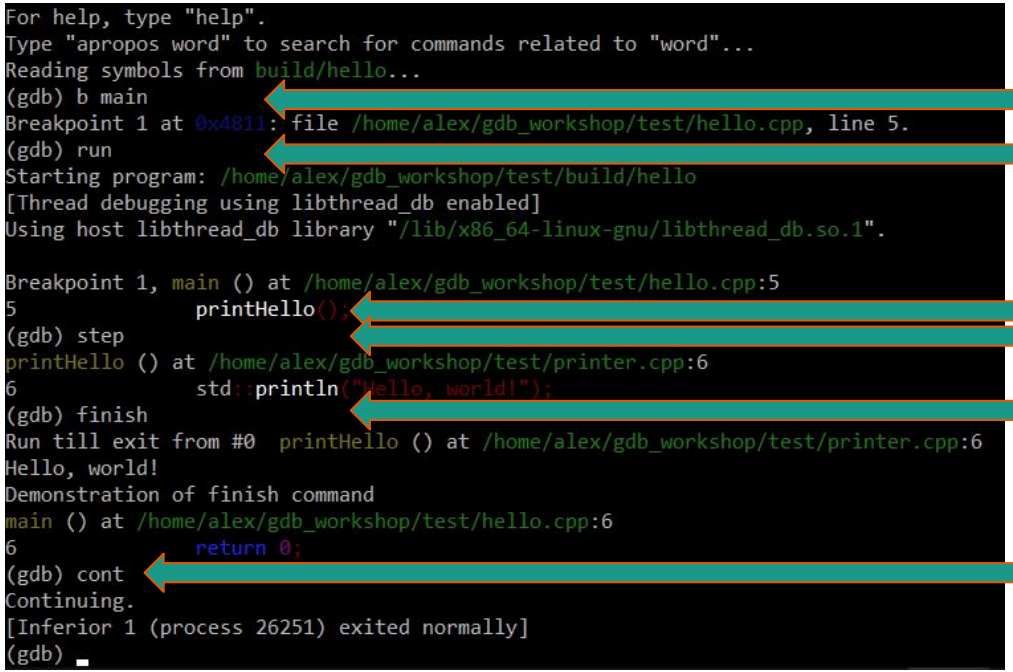
Run the program

Breakpoint hit

Step into the printHello() function

Step out of the printHello() function

Basic GDB usage demo - explained



Set a breakpoint in main()

Run the program

Breakpoint hit

Step into the printHello() function

Step out of the printHello() function

Continue the program execution

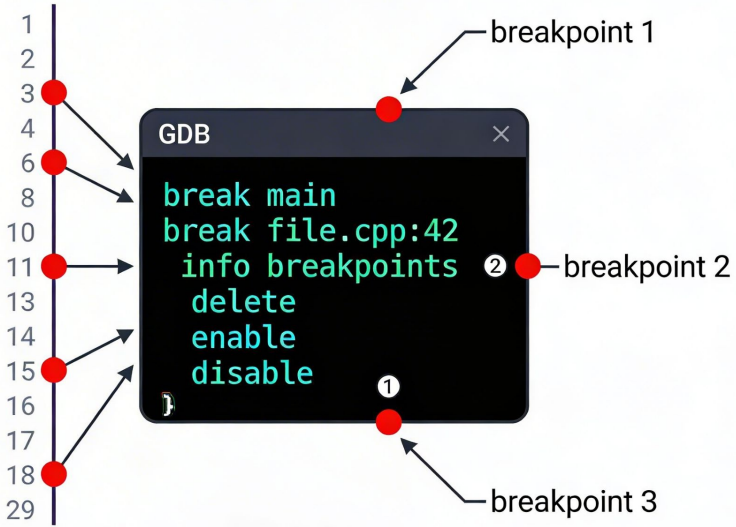
Providing arguments to the program

- Start debug session - **`gdb <program>`**
- Supply arguments in GDB CLI
  - **`run [args]`** - start the program with optional arguments
  - **`run < input.txt`** - provide stdin from file
  - **`run 2>errors.log`** - Redirect stderr to file
  - **`set args arg1 arg2 ... argN`** - Set arguments persistently
  - **`show args`** - Display current arguments

Debug information - The key for success!

- No debug symbols means no names, no lines, and barely any story - just addresses and guesswork.
- Debug symbols are the map from machine code back to source code: functions, variables, types, files, and line numbers.
- Make backtraces possible, because unwinding and frame reconstruction depend heavily on debug information such as DWARF.

Breakpoints



What Are Breakpoints?

- Pause the program execution at a specific location
- Allows the programmer to inspect memory, variables and call stack
- Several types of breakpoints exist
  - **Line breakpoint**: A breakpoint at a predefined line (and file)
  - **Conditional breakpoint**: A breakpoint that pauses execution only if a specific condition is met
  - **Data breakpoint (Watchpoint)**: Pauses when a specific memory address is read or modified

Setting Breakpoints

- **`break file.cpp:42`** - At line 42 in file.cpp
- **`break function_name`** - At function_name entry
- **`break Class::MemberFunction`** - As above, but at member function
- **`break *0x00001234`** - At a specific function address (not variables!)

Managing breakpoints

- **`info breakpoints`** - List all breakpoints with IDs
- **`disable <breakpoint-id>`** - Disable breakpoint temporarily
- **`enable <breakpoint-id>`** - (Re–)enable the breakpoint
- **`delete <breakpoint-id>`** - Permanently remove the breakpoint
- **`clear file.cpp:42`** - Delete all breakpoints at line 42 of file.cpp
- **`set breakpoint pending on/off`** - Set in not yet loaded libraries

Breakpoints - Example

`day01/01_breakpoints/01_basic_example`

Managing breakpoints - commands

Actions on breakpoint hit (similar to “breakpoint actions” in some IDEs)

```
commands <breakpoint-id>
> <action1>
> ...
> <action N>
> end
```



Commands - Example

`day01/01_breakpoints/03_commands`

Commands - Example Details

The code: day01/01_breakpoints/03_commands/main.cpp

The script: day01/01_breakpoints/03_commands/commands.gdb

Breakpoints in overloaded functions

What happens when a breakpoint is set in an overloaded function?

Breakpoints in overloaded functions

What happens when a breakpoint is set in an overloaded function?

`break foo` ⇒ can be ambiguous, so GDB may find several matching locations.

GDB command **`set multiple-symbols <VALUE>`** can toggle the working mode.

- **`all`** - set on all matches (default behavior)
- **`ask`** - GDB asks what to do
- **`cancel`** - Cancels the command if there is more than 1 match

The safest option is to name the exact overload, e.g. **`break foo(int)`**

Breakpoints in templated functions

```
(gdb) b MyFunc<T>
```

Overload info:

```
(gdb) info functions PrintValue
 All functions matching regular expression "PrintValue":

 File /home/alex/gdb_workshop/day01/01_breakpoints/04_templates/main.cpp:
 7:      void PrintValue<double>(double const&);
 7:      void PrintValue<int>(int const&);
 7:      void PrintValue<std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >
>(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&);
```

Breakpoints in templated functions

The same goes for classes:

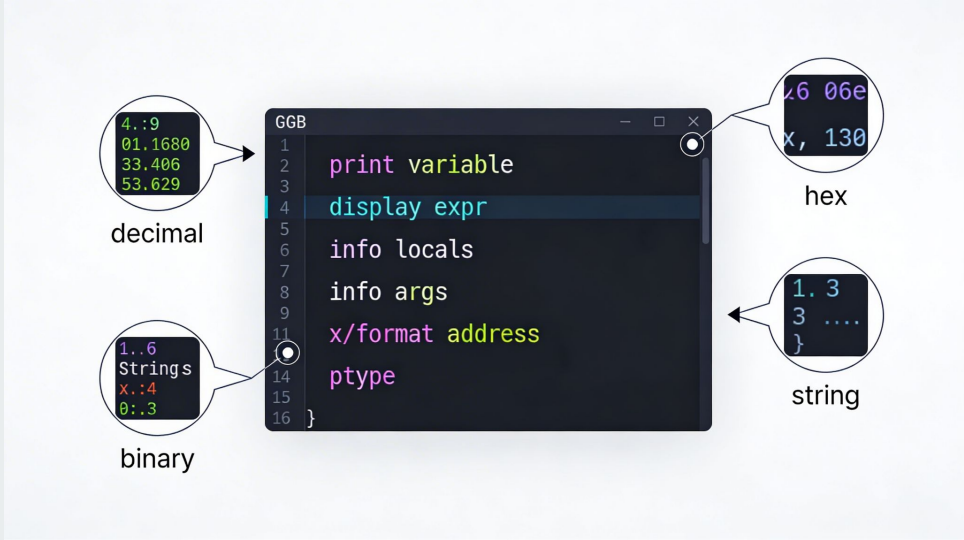
```
(gdb) info functions Calculator
 All functions matching regular expression "Calculator":

 File /home/alex/gdb_workshop/day01/01_breakpoints/04_templates/main.cpp:
 18:     void Calculator<double>::Add(double);
 16:     void Calculator<double>::Calculator(double);
 18:     void Calculator<int>::Add(int);
 16:     void Calculator<int>::Calculator(int);
```

Breakpoints - Example

`day01/01_breakpoints/02_functions_overload`

Data display in GDB



Why do we need to display data in GDB?

- **State verification at execution points** - Exposes unexpected state changes
- **Root cause identification** - Inspecting data helps trace errors to their source
- **Reduced time-to-resolution** - Data inspection eliminates guesswork and accelerates issue diagnosis
- **Pattern and anomaly detection** - Reveals recurring issues that are easily missed in raw text logs

How to display?

- **`info locals`** - Show information about local variables
- **`info args`** - Show information about function arguments
- **`print variable`** - Show variable value (auto-formatted)
- **`print *ptr`** - Dereference pointer
- **`print *arr@N`** - Print array range (start at `arr` and print `N` entries)
- **`set print array on`** - Pretty-print arrays (one element per line)
- **`set print pretty on`** - Specifically affects classes, structures and unions
- **`x 0x7fff1000`** - Examine memory at address (hex dump)

Some more display techniques

- **`display variable`** - Auto-print at each breakpoint
- **`info display`** - List active displays
- **`undisplay 1`** - Remove display #1
- **`whatis variable`** - Show variable type
- **`ptype MyClass`** - Show class structure and methods

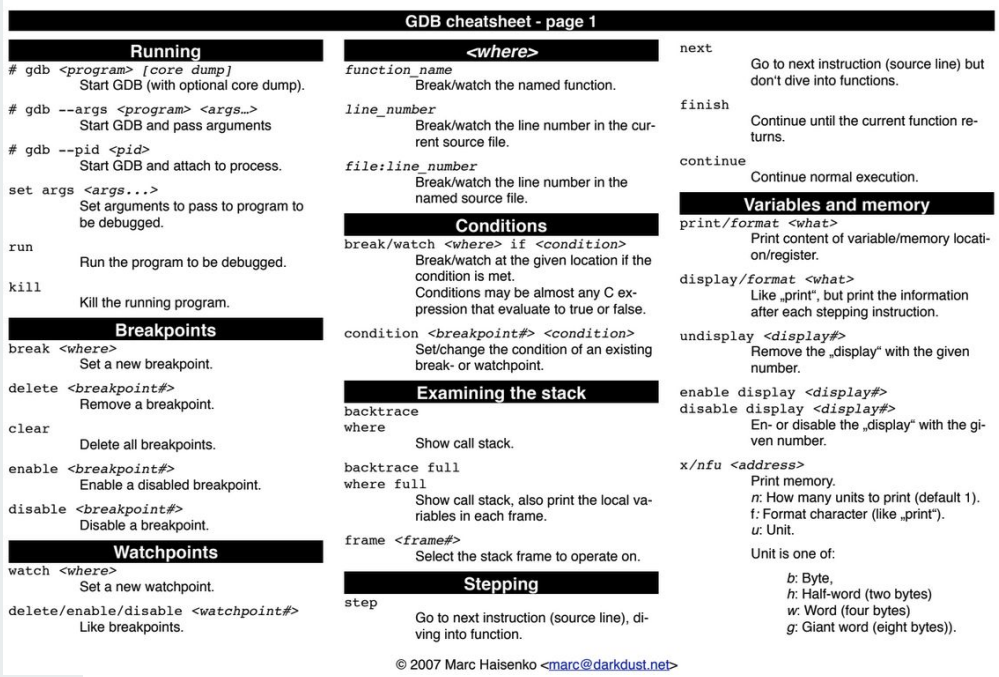
Exercise


(gdb)
Breakpoint 1 at 0x1234:
file main.c, line 10
Program received signal SIGINT,
Interrupt.
Try 'step' to execute line by line
Use 'print var' to inspect variables
Set breakpoints with 'break function'

Exercise

exercises/day01/ex01/README.md

GDB Cheatsheet



GDB cheatsheet - page 1

Running

`# gdb <program> [core dump]`
Start GDB (with optional core dump).

`# gdb --args <program> <args…>`
Start GDB and pass arguments

`# gdb --pid <pid>`
Start GDB and attach to process.

`set args <args...>`
Set arguments to pass to program to be debugged.

`run`
Run the program to be debugged.

`kill`
Kill the running program.

Breakpoints

`break <where>`
Set a new breakpoint.

`delete <breakpoint#>`
Remove a breakpoint.

`clear`
Delete all breakpoints.

`enable <breakpoint#>`
Enable a disabled breakpoint.

`disable <breakpoint#>`
Disable a breakpoint.

Watchpoints

`watch <where>`
Set a new watchpoint.

`delete/enable/disable <watchpoint#>`
Like breakpoints.

<where>

`function_name`
Break/watch the named function.

`line_number`
Break/watch the line number in the current source file.

`file:line_number`
Break/watch the line number in the named source file.

Conditions

`break/watch <where> if <condition>`
Break/watch at the given location if the condition is met.
Conditions may be almost any C expression that evaluate to true or false.

`condition <breakpoint#> <condition>`
Set/change the condition of an existing break- or watchpoint.

Examining the stack

`backtrace`
`where`
Show call stack.

`backtrace full`
`where full`
Show call stack, also print the local variables in each frame.

`frame <frame#>`
Select the stack frame to operate on.

Stepping

`step`
Go to next instruction (source line), diving into function.

`next`
Go to next instruction (source line) but don't dive into functions.

`finish`
Continue until the current function returns.

`continue`
Continue normal execution.

Variables and memory

`print/format <what>`
Print content of variable/memory location/register.

`display/format <what>`
Like „print“, but print the information after each stepping instruction.

`undisplay <display#>`
Remove the „display“ with the given number.

`enable display <display#>`
`disable display <display#>`
En- or disable the „display“ with the given number.

`x/nfu <address>`
Print memory.
n: How many units to print (default 1).
f: Format character (like „print“).
u: Unit.

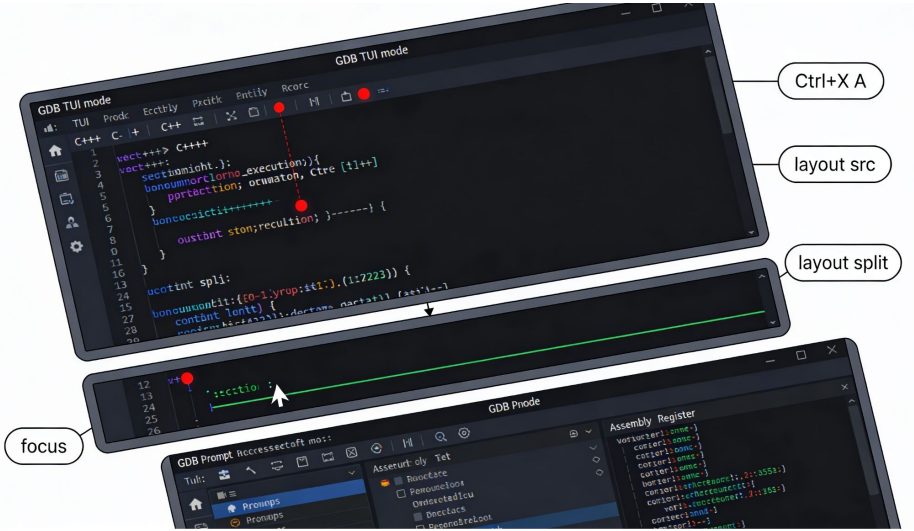
Unit is one of:

b: Byte,
h: Half-word (two bytes)
w: Word (four bytes)
g: Giant word (eight bytes)).

© 2007 Marc Haisenko <marc@darkdust.net>

https://darkdust.net/files/GDB%20Cheat%20Sheet.pdf

Using GDB with TUI



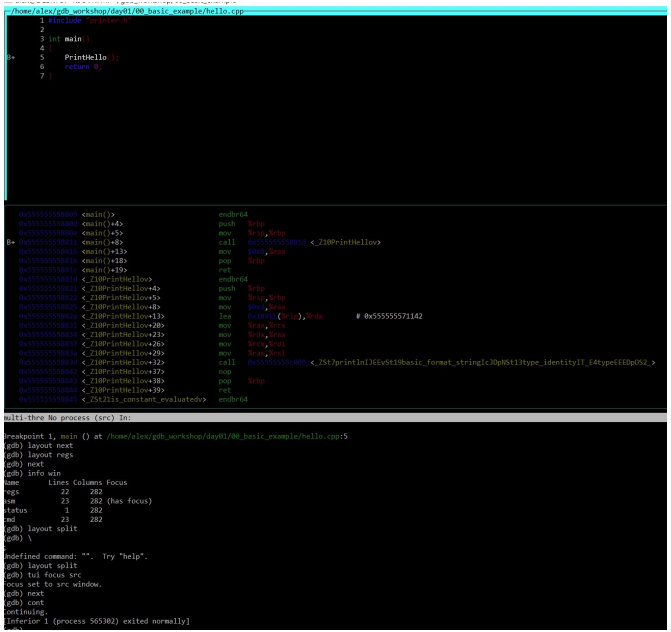
What is TUI? And how to use it?

TUI (Text User Interface) is a curses-based terminal view with windows for code, assembly, registers, and commands, making debugging more visual

What is TUI? And how to use it?

- **`gdb --tui <program>`** or **`tui enable`** from the **GDB** shell or **`Ctrl+X A`**
- Several layouts available
  - **`layout src/asm/regs`** - Displays the source code, disassembly and registers
  - **`layout prev/next`** - Cycle layouts
- **`tui focus <layout>`** - Focuses on the selected layout
- **`tui new-layout newlayout src 1 regs 1 status 0 cmd 1`** - Creates a new layout

TUI layout example



GDB TUI - demo

`day01/00_basic_example`

Exercise


(gdb)
Breakpoint 1 at 0x1234:
file main.c, line 10
Program received signal SIGINT,
Interrupt.
Try 'step' to execute line by line
Use 'print var' to inspect variables
Set breakpoints with 'break function'

Exercise

exercises/day01/ex02/README.md

Part 1 Summary

- Introduction
- Basic GDB CLI usage and commands
- Program data display
- Breakpoints
- GDB TUI

Towards Part 2

- Debug symbols
- VSCode and GDB
- Advanced breakpoints
  - Conditional
  - Watchpoints
  - Catchpoints
- Practical exercise

Lunch break


GDB gdb (GNU) 11.2
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <ht
...
(Attaching to process 12345)
Reading symbols from ./data_analyser...done.
[New Thread 12345.12347]
(gdb) watch shared_data->count
Hardware watchpoint 1: shared_data->count
(gdb) watch *(shared_data->data+1234)
Hardware watchpoint 2: *(shared_data->data+123
(gdb) _



GDB debugging workshop

Session 2

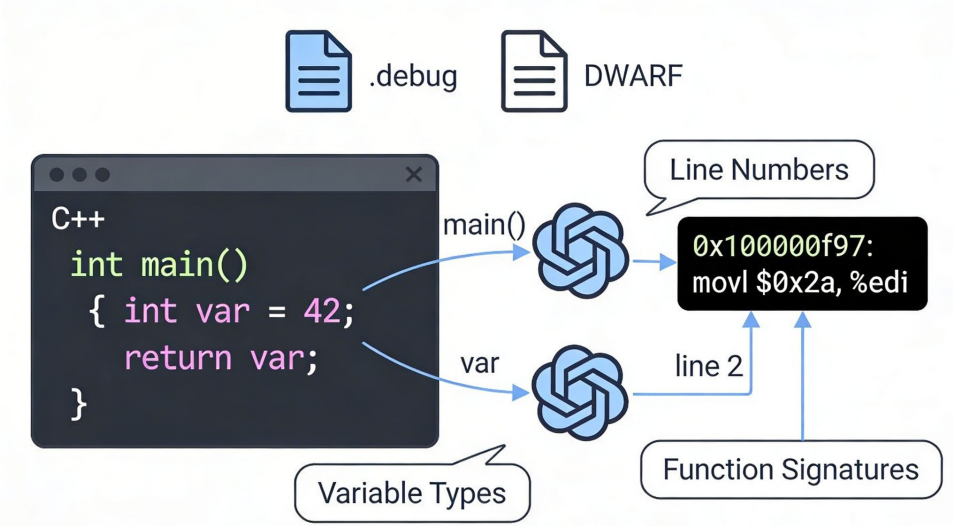
Part 1 Recap

- Introduction
- Basic GDB CLI usage and commands
- Program data display
- Breakpoints
- GDB TUI

Part 2

- Debug symbols
- VSCode and GDB
- Advanced breakpoints
  - Conditional
  - Watchpoints
  - Catchpoints
- Practical exercise

More on debug symbols

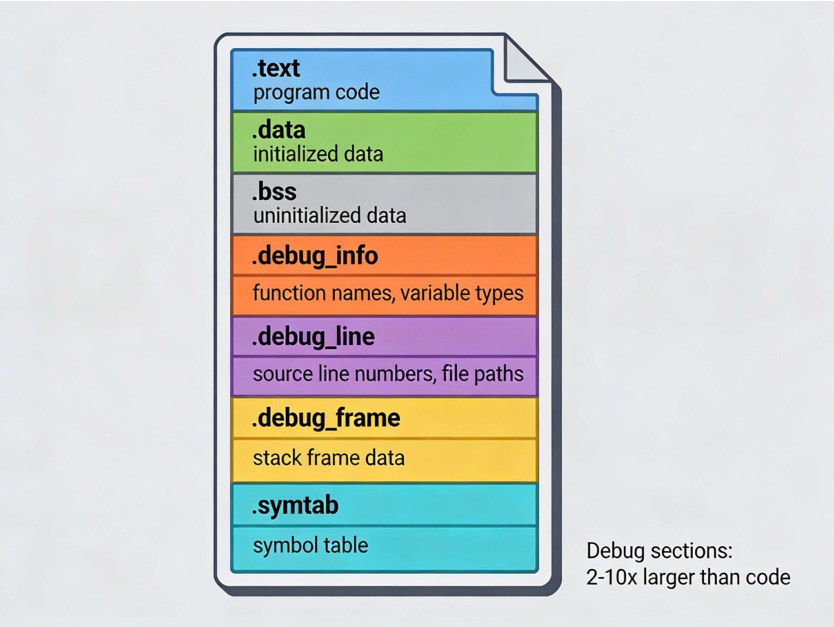


DWARF format

- Debug data is stored in **DWARF** (“Debugging With Attributed Record Formats”)
- Originated along with **ELF** binary format
- Latest standard (DWARF5) was published in 2017
- Key sections include (but not limited to)
  - .debug_info (core symbols and types)
  - .debug_abbrev (structure definitions),
  - .debug_line (source line mappings), and .debug_str (strings)

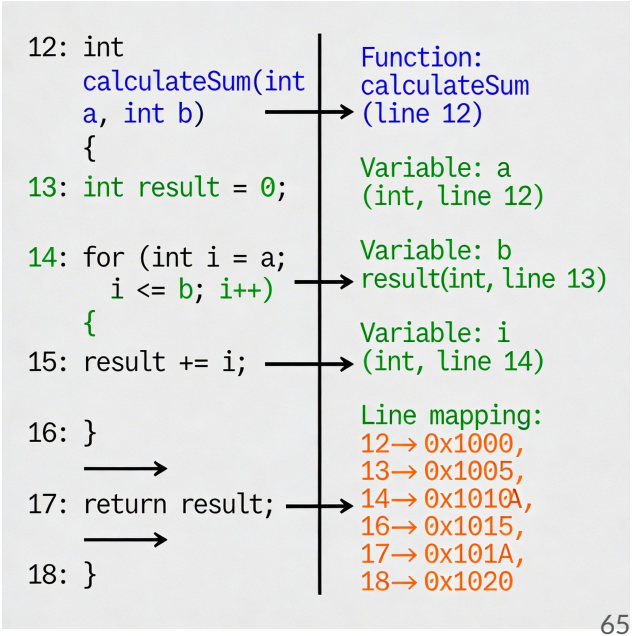
How debug symbols relate to a program?

Compiled with -g



Symbol sections content

- Metadata mapping machine to source code locations
- Embedded into the executable by default
- Can be stripped and saved in a separate file
- Contains function names, types, line numbers, etc.



Displaying the symbol table

On Linux `nm`, `objdump` or `readelf` are used to inspect an ELF executable symbol table.

Tool	Primary Use	Symbol Flag	Other Features	Best For
`nm`	List symbols	(default), `-D` dynamic	`-C` demangle	Quick symbol lookup
`objdump`	Disassembly + headers	`-t` full, `-T` dynamic	`-d` disassembly `-x` headers `-C` demangle	Code analysis, sections
`readelf`	ELF structure details	`-s` (both tables), `--dyn-syms` dynamic	Headers, sections, program headers	ELF format inspection

readelf example output

Num: Symbol index in table.
Value: Virtual address.
Size: Symbol size (bytes).
Type: FUNC/OBJECT/NOTYPE/SECTION/etc.
Bind: LOCAL/GLOBAL/WEAK.
Vis: DEFAULT/HIDDEN/PROTECTED.
Ndx: Section index (UND=undefined, ABS=absolute, COM=common).
Name: Symbol name

```
Symbol table '.symtab' contains 851 entries:
   Num:    Value          Size Type    Bind   Vis      Ndx Name
     0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
     1: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS Scrt1.o
     2: 000000000000038c    32 OBJECT  LOCAL  DEFAULT    4 __abi_tag
     3: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS crtstuff.c
     4: 0000000000004750     0 FUNC    LOCAL  DEFAULT   16 deregister_tm_clones
     5: 0000000000004780     0 FUNC    LOCAL  DEFAULT   16 register_tm_clones
     6: 00000000000047c0     0 FUNC    LOCAL  DEFAULT   16 __do_global_dtors_aux
     7: 0000000000027020     1 OBJECT  LOCAL  DEFAULT   28 completed.0
     8: 0000000000026a40     0 OBJECT  LOCAL  DEFAULT   23 __do_global_dtor[...]
     9: 0000000000004800     0 FUNC    LOCAL  DEFAULT   16 frame_dummy
    10: 0000000000026a38     0 OBJECT  LOCAL  DEFAULT   22 __frame_dummy_in[...]
    11: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS main.cpp
    12: 000000000001d2c0     1 OBJECT  LOCAL  DEFAULT   18 _ZNSt8__detail30[...]
    13: 000000000001d2c1     1 OBJECT  LOCAL  DEFAULT   18 _ZNSt8__detail30[...]
    14: 000000000001d2c2     1 OBJECT  LOCAL  DEFAULT   18 _ZNSt8__detail30[...]
    15: 000000000001d2c3     1 OBJECT  LOCAL  DEFAULT   18 _ZSt39__is_impli[...]
    16: 000000000001d2c4     1 OBJECT  LOCAL  DEFAULT   18 _ZSt39__is_impli[...]
    17: 000000000001d2c5     1 OBJECT  LOCAL  DEFAULT   18 _ZNSt8__detail14[...]
    18: 000000000001d2c6     1 OBJECT  LOCAL  DEFAULT   18 _ZNSt8__detail19[...]
```



Calling functions from external libraries

How does an executable know the function names from an external shared library?

Calling functions from external libraries

How does an executable know the function names from an external shared library?

The executable contains **undefined symbols** for external functions:

- Marked as **U** in nm output
- Reference function names only, not addresses
- Resolved at runtime by ld.so (dynamic linker)
- Even if the library is stripped from symbols, ld can still resolve the symbols
- Only the **debug symbol table** is stripped, while the **dynamic symbol table** still persists

A word about “plugins”

- **Plugin architecture** - .so loaded at runtime can call functions from the main elf.
- **Symbol visibility** - By default, main executable's symbols are **not exported** to plugins.
- **Enable access** - Compile main with `-rdynamic` flag to export all symbols to dynamic symbol table.
- **Runtime resolution** - Plugin resolves main's symbols via `dlsym(RTLD_DEFAULT, "func_name")` or direct calls if linked properly
- **Symbol lookup** - Dynamic linker searches main executable's global symbol table when plugin calls undefined symbols

Plugins - example

`day01/08_plugins`

Optimization flags when debugging

There are various considerations regarding optimization level when debugging.

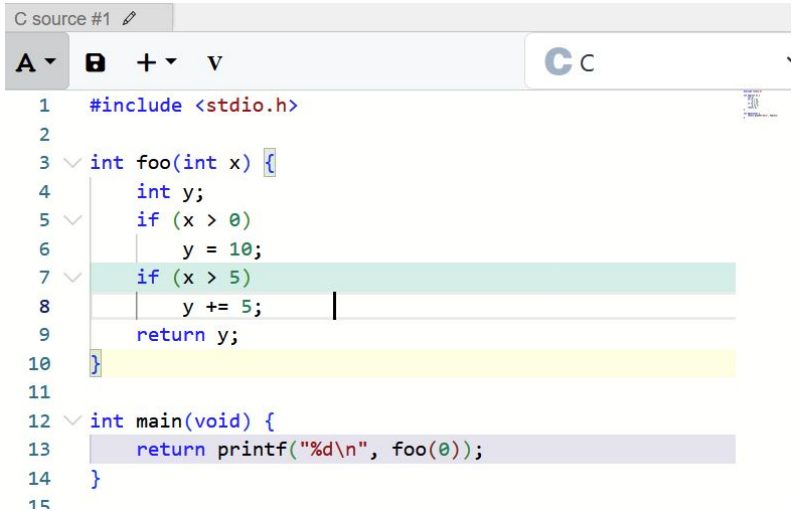
- Debug the production build (optimized)
  - May be "Weird": the debugger behaves weird relative to your source
- Debug an unoptimized build (-O0, matches your mental model)
  - May be "Weird": may behave differently than production

Comparison between optimization levels

Aspect	-O0	-Og	-O2 / -O3
Purpose	No optimization (default)	Debug-friendly optimization	Optimization for production
Debug experience (with -g)	Full source-line mapping; all locals visible	Close to -O0	Distorted: code rearrangement, optimized out variables
Optimizations	None (passes disabled)	Basic -O1 minus debug-interfering flags	Aggressive: inlining, unrolling, vectorization
Performance	Slowest	Moderate speedup	Fastest
Use Case	Pure debugging	Edit-compile-debug cycle	Closest to production

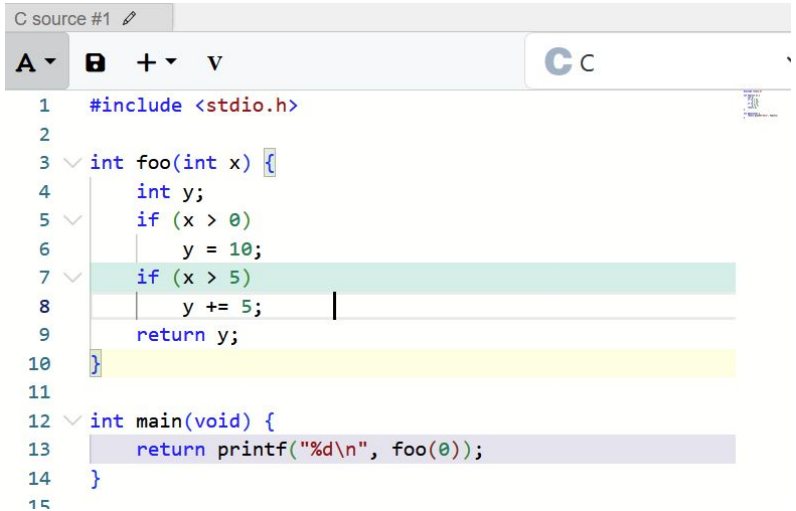
When -O0 misleads

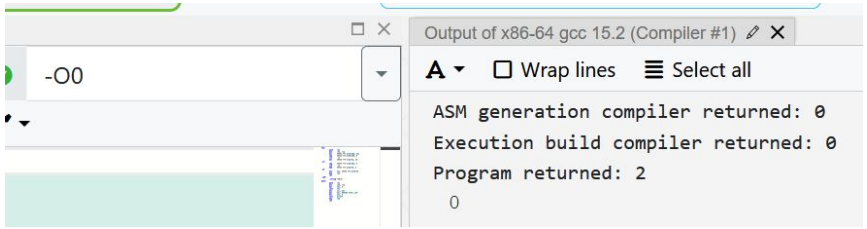
https://godbolt.org/z/7oeoEqfjo



When -O0 misleads

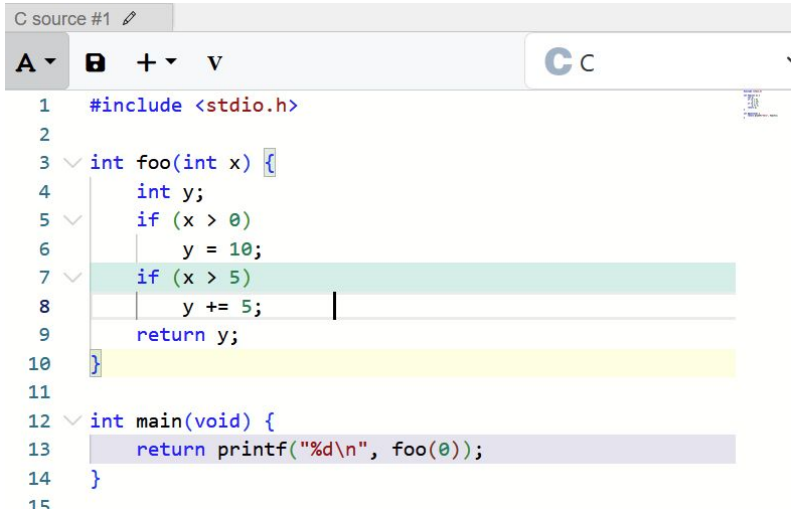
https://godbolt.org/z/7oeoEqfjo

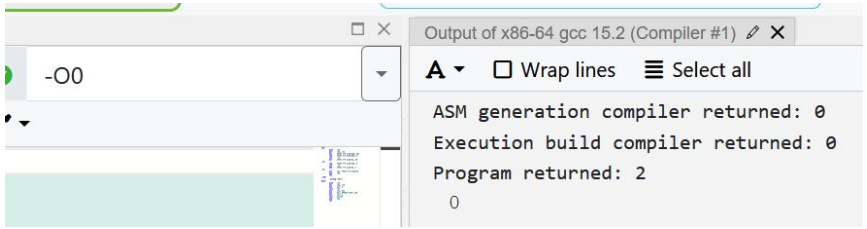


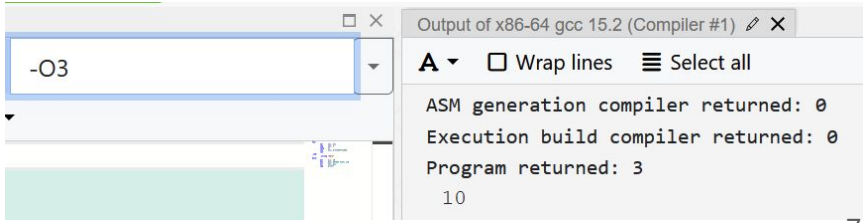


When -O0 misleads

https://godbolt.org/z/7oeoEqfjo







On the other side…

https://lwn.net/Articles/342330/

```
static unsigned int tun_chr_poll(struct file *file, poll_table * wait)
{
    struct tun_file *tfile = file->private_data;
    struct tun_struct *tun = __tun_get(tfile);
    struct sock *sk = tun->sk;
    unsigned int mask = 0;

    if (!tun)
        return POLLERR;
```

On the other side…

https://lwn.net/Articles/342330/

```
static unsigned int tun_chr_poll(struct file *file, poll_table * wait)
{
    struct tun_file *tfile = file->private_data;
    struct tun_struct *tun = __tun_get(tfile);
    struct sock *sk = tun->sk;
    unsigned int mask = 0;

    if (!tun)
        return POLLERR;
```

On the other side...

https://lwn.net/Articles/342330/

Sometimes it is possible that **tun** will be NULL and **sk** will be still valid

```
static unsigned int tun_chr_poll(struct file *file, poll_table * wait)
{
    struct tun_file *tfile = file->private_data;
    struct tun_struct *tun = __tun_get(tfile);
    struct sock *sk = tun->sk;
    unsigned int mask = 0;

    if (!tun)
        return POLLERR;
```

On the other side…

https://lwn.net/Articles/342330/

Sometimes it is possible that **tun** will be NULL and **sk** will be still valid

But, gcc assumed that since **sk** was already dereferenced, **tun** cannot be NULL, and there is no point to check it

```
static unsigned int tun_chr_poll(struct file *file, poll_table * wait)
{
    struct tun_file *tfile = file->private_data;
    struct tun_struct *tun = __tun_get(tfile);
    struct sock *sk = tun->sk;
    unsigned int mask = 0;

    if (!tun)
        return POLLERR;
```

Compiler flags for additional debug information

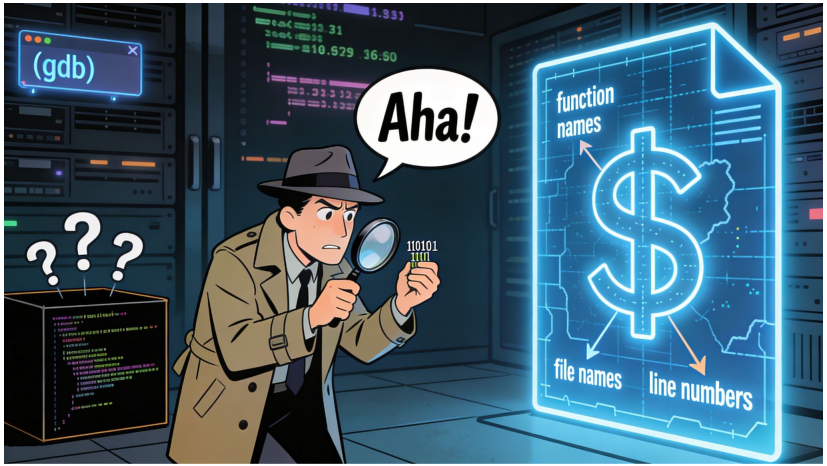
- **`-g`** - Include standard debug symbols (GCC/Clang default)
- **`-ggdb3`** - Extended symbols including macro information (can be also 1/2/3)
- **`-fno-omit-frame-pointer`** - Preserve frame pointers for reliable backtraces

Self-play example

`day01/03_preparing_the_executable`

- Configure the build with
  - **`-DUSE_GGDB3=ON`**
  - **`-DUSE_NO_OMIT_FRAME_POINTER=ON`**
  - **`-DCMAKE_BUILD_TYPE=Debug`**
- Build the program
- Debug the program - observe macro expansions and frame pointers

Symbol files and production code





Symbol files and production code

Would we like to release our product as is, with the debug information?

Symbol files and production code

Would we like to release our product as is, with the debug information?

We may want to “obfuscate” the binary code

- Expose no hints about variables and function names
- Do not expose internal paths (code location, build directories)

Stripping Debug Information

- Done by “stripping” the debug symbols from the executable
- Debug info can be stored in safe place, before it is stripped
- Stripping also significantly reduces the binary size
- When a field case arrives (for example a core dump), symbols can be loaded into the debug session

Debug Fission

Debug fission, also known as DWARF fission or split DWARF, separates debugging information from the main executable or object files during compilation.

Debug Fission (1)

- DWARF extension that "split" debug information into separate files.
- Reducing binary size and improving debug performance.
- Especially relevant for GDB workflows in large C and C++ projects.

Debug Fission (2)

- Fission offloads the DWARF debug info (**`.debug_*`** sections) to sidecar **`.dwo`** or **`.dwp`** files
- The binary retains only minimal "skeleton" info (addresses, abbreviations, hashes) to reference them.
- Achieved with **`-gsplit-dwarf`** compiler flag
- GDB then searches for the .dwo files in predefined paths
- **`show/set debug-file-directory`** from GDB shell

SKIP

Debug fission - Demo

- `day01/03_preparing_the_executable/04_fission`

Indexing

The problem:

GDB scans all DWARF files linearly on startup

In large projects, it can reach 30-60 seconds

The solution:

Create a section embedded in the exe (.gdb_index) containing symbol lookup table

The method:

Pass to the compiler the `-Wl,--build-id --gdb-index` flag **OR**

Built-in GDB command `save gdb-index <dir>` **OR**

External tool `gdb-add-index <prog>`

Indexing

The problem:

GDB scans all DWARF files linearly on startup to build symbol tables

In large projects, it can reach 30-60 seconds

The solution:

embed an index section (.gdb_index, or DWARF 5 .debug_names) so gdb skips the full parse.

Methods (any one):

Compile with `-ggnu-pubnames [-gdwarf-5]` (or `-gsplit-dwarf`, which implies it) and link with `-Wl,--gdb-index` (needs `gold/lld/mold`)

GDB command: `(gdb) save gdb-index <dir>`

External tool: `gdb-add-index [--dwarf-5] <prog> –` generates and embeds; works on any -g binary

Exponential boost

Project size (files)	Load time without index	Load time with index
Small (~5-6)	0.2s	0.1s
Medium (~50 files)	5s	0.5s
Large (~500+ files)	45s	2s

Real-life problem: Project versions and symbols

Version mismatches break debugging:

Binary compiled with one toolchain/library version while debugger loads symbols from different version causes incorrect variable locations, type definitions, and stack unwinding.

How can we handle symbols in scale?

Solution: symbol server

- Eliminates the need to distribute the software with symbols.
- Automatic symbols download according to build ID (embedded into the executable).
- Shared across development teams.
- May provide local caching for performance improvement.

Symbols server in Microsoft world (may skip)

- **Format:** PDB (Microsoft-specific)
- **Tools:** WinDbg, Visual Studio, CLion
- **Protocol:** HTTP/HTTPS, UNC (File share)
- **Servers:** Free symbols for Windows OS and Microsoft products, additional servers can be added

Symbols server - Linux world

Modern distributions provide the ***debuginfod*** utility

- **Format:** DWARF
- **Tools:** GDB, LLDB
- **Protocol:** HTTP/HTTPS, local file storage
- **Repositories:** ddebs.ubuntu.com, debuginfod.elfutils.org
- Custom repositories can be added to **`DEBUGINFOD_URLS`** env. variable

SKIP

Symbols server - debuginfod - Demo

- `day01/09_debuginfod`

Stripping the symbols - Manual flow (less common)

Stripping can be done manually, without fission (old-style, less common today).

Manual stripping without debug fission involves a post-build workflow where debug symbols are extracted from the fully-linked executable into separate .debug files, then removed from the binary, and finally reconnected via embedded references. This creates smaller release binaries while preserving debuggability through external symbol packages that debuggers can locate automatically.

Manually stripping the symbols (1)

- Strip only debug sections, but leave the symbol table
  - `objcopy --only-keep-debug <artifact> <debug-symbols>`
  - `objcopy --strip-debug <artifact>`
  - `objcopy --add-gnu-debuglink=<debug-symbols> <artifact>` - Automatically connect the symbol file with the artifact
  - `objcopy --strip-all <artifact>` - for even smaller executable - harder to field-debug

Manually stripping the symbols (2)

Scenario	Flag	Why
Workshop/Learning	`--strip-all`	Makes very obvious that symbols are missing, forcing to learn how to load
Production software	`--strip-debug`	Keeps the binary small, but allows for “symbolic” stack traces in crash logs
Strict security	`--strip-all`	Function names unknowable without the symbol file.

CLI/TUI? What are we, in 1991?



UI for everybody!

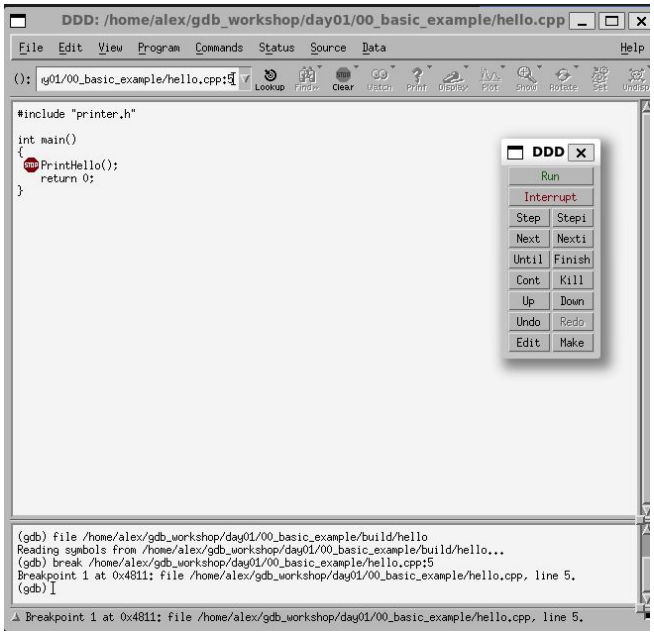


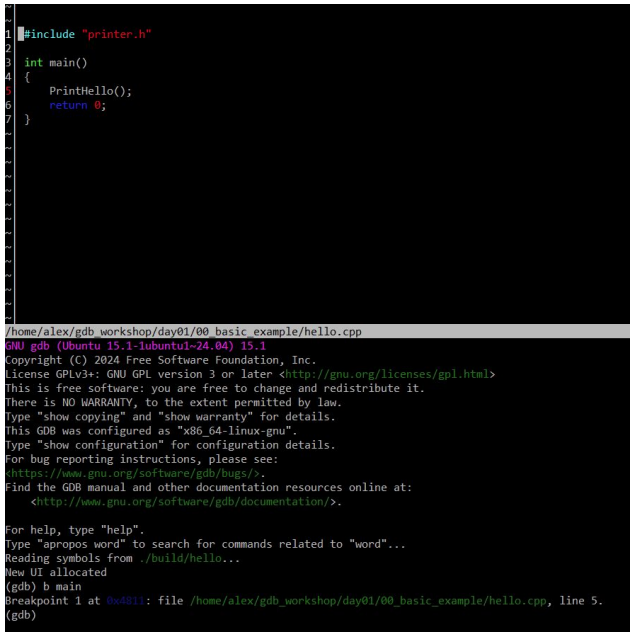
GDB frontends (1)

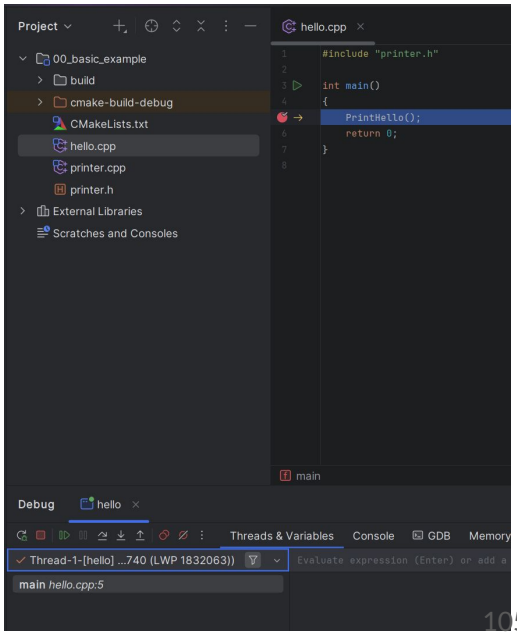
Several GDB frontends exist

- IDEs - VSCode, CLion, Eclipse
- Standalone - DDD, GDB dashboard, cgdb

GDB frontends (2)







Connecting GDB to VSCode

- In VSCode, debugging is configured by **`launch.json`** file, stored in **`.vscode`** folder
- Defines launch configurations with key properties like **`type`**, **`request`**, **`name`**, **`program`**
- How to create
  - Open a C++ source file
  - Hit **`Ctrl + Shift + D`**
  - Click on **“`create a launch.json file`”**
  - Select the configuration (launch/attach/etc.)

launch.json example



```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "(CMake) Debug",
            "type": "cppdbg",
            "request": "launch",
            "program": "${command:cmake.launchTargetPath}",
            "args": [
                "${input:programArgs}"
            ],
            "stopAtEntry": false,
            "cwd": "${workspaceFolder}",
            "environment": [],
            "externalConsole": false,
            "MIMode": "gdb",
            "miDebuggerPath": "/usr/bin/gdb",
            "setupCommands": [
                {
                    "description": "Enable pretty-printing for GDB",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                },
                {
                    "description": "Set Disassembly Flavor to Intel",
                    "text": "-gdb-set disassembly-flavor intel",
                    "ignoreFailures": true
                }
            ]
        }
    ],
    "inputs": [
        {
            "id": "programArgs",
            "description": "Enter arguments (space-separated)",
            "type": "promptString",
            "default": ""
        }
    ]
```

launch.json example

```
"request": "launch",
"program": "${command:cmake.launchTargetPath}",
"args": [
    "${input:programArgs}"
],
```



launch.json example



Generating a launch file using LLM

I have a CMake-based project. I work with VSCode, and I want to setup a debugger. I use gdb, and I need to generate a launch.json

Generating a launch file using LLM

I have a CMake-based project. I work with VSCode, and I want to setup a debugger. I use gdb, and I need to generate a launch.json

To set up GDB debugging for your CMake project in VSCode, install the C/C++ and CMake Tools extensions if not already done. Ensure your CMake build uses Debug mode (e.g., `-DCMAKE_BUILD_TYPE=Debug`) to include symbols, then create a `.vscode/launch.json` file with a configuration that leverages CMake Tools variables for the executable path.

`code.visualstudio +1`

Prerequisites

- CMake Tools extension detects your build targets automatically.
- Select a launch target via the status bar (Debug button) after building once.
- GDB must be installed (standard on Linux like your Tel Aviv setup). `code.visualstudio`

Generating a launch file using LLM

Sample launch.json

Place this in `.vscode/launch.json`. It uses `${command:cmake.launchTargetPath}` to dynamically point to your selected CMake target's executable. `stackoverflow`

```json
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "(gdb) Launch",
            "type": "cppdbg",
            "request": "launch",
            "program": "${command:cmake.launchTargetPath}",
            "args": [],
            "stopAtEntry": false,
            "cwd": "${workspaceFolder}",
            "environment": [],
            "externalConsole": false,
            "MIMode": "gdb",
            "setupCommands": [
                {
                    "description": "Enable pretty-printing for gdb",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                }
            ]
```

Launch Correctly

- Run/Debug view: Dropdown > **select your config name** (e.g., "CMake Debug Args") > F5/green play. `stackoverflow +1`
- Wrong: "Debug anyway" or top F5 button skips `launch.json`. `github`
- Prompt appears before launch; enter "arg1 arg2", OK.

Troubleshooting

Issue	Fix
No prompt	Select config from dropdown; reload `stackoverflow`
"Inputs not allowed"	Root-level `"inputs"`; valid JSON `stackoverflow`
Empty args	Use `["${input:programArgs}"]` (splits spaces) `github`
Multi-args	Prompt handles spaces; or `["arg1", "${input:arg2}"]` `code.visualstudio`

Let's start debugging with VSCode!

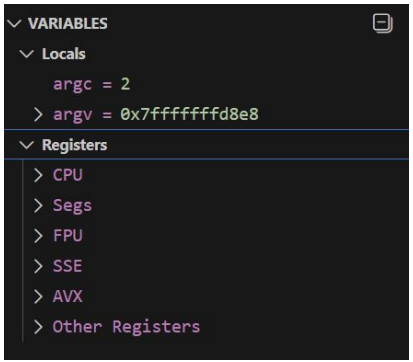




Start debugging

- Set the desired breakpoints
- Press F5
- Most of the shortcuts are like in Visual Studio (F5, F10, F11, etc.)



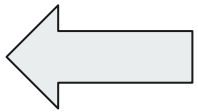


So why use CLI at all?

- Sometimes, especially in the field, there will be no UI
- Not all GDB capabilities are implemented in all frontends
- Some useful features absent from VSCode
  - Catchpoints
  - Reverse debugging
  - Scripted commands
  - And more

So why use CLI at all?

- Sometimes, especially in the field, there will be no UI
- Not all GDB capabilities are implemented in all frontends
- Some useful features absent from VSCode
  - Catchpoints
  - Reverse debugging
  - Scripted commands
  - And more



We'll see it later today

Exercise


(gdb)
Breakpoint 1 at 0x1234:
file main.c, line 10
Program received signal SIGINT,
Interrupt.
Try 'step' to execute line by line
Use 'print var' to inspect variables
Set breakpoints with 'break function'

Exercise

exercises/day01/ex03/README.md

Advanced breakpoint features

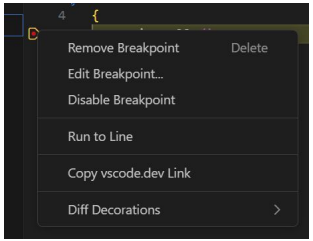


Conditional breakpoints

- Pauses program execution only when a specific condition is met
  - A boolean user-defined condition evaluates to true
  - A hit count threshold is met
- Crucial for debugging loops or frequently hit code blocks
- Supported in most modern IDEs

Conditional breakpoints in VSCode

Right-click on the breakpoint, then “Edit breakpoint”




```
3  int main(int argc, char* argv[])
4  {
5      PrintHello();
```

Expression — Break when expression evaluates to true. 'Enter' to accept, 'Escape' to cancel.

```
6      return 0;
7  }
```

Can choose expression, hit count or dependency on another breakpoint.
In addition, can log to console a custom message.

Conditional breakpoints in CLI

- Information about the breakpoints can be retrieved by **`info breakpoints`** command
- The condition is set using the breakpoint ID
  - **`condition <ID> counter == 100`** - Pause when counter variable equals 100
  - **`ignore <ID> 5`** - Ignore breakpoint ID 5 times
  - **`commands <ID> / print var / continue / end`** - Actions on breakpoint hit
- New conditional breakpoint: **`break file.cpp:123 if condition == true`**
- **`set break pending on`** - Enable setting breakpoints in not-yet-loaded libraries



Advanced breakpoints example

day01/05_advanced_breakpoints

Watchpoints

- Halts execution whenever the value of a specific **expression or variable** changes.
- Can be configured to trigger on different types of memory access.
    - **Hardware:** Use dedicated CPU debug registers to monitor an address in hardware, with negligible runtime overhead; limited in number
    - **Software:** GDB's fallback if hardware registers are full or unsupported; significantly slower
- A watchpoint can only be set on a variable that is currently in scope
- Essential for debugging memory corruption or "mystery" bugs where a pointer is being modified by an unknown part of the codebase.

A practical example

```
10  void ProcessData(std::vector<Data>& data)
11  {
12      for (size_t i = 0; i < data.size(); ++i)
13      {
14          data[i].m_value += 1;
15
16          if (i == 10)
17          {
18              std::println("Processing index 10...");
19              data[42].m_value = -9999;
20          }
21      }
22  }
```

```
int main()
{
    std::vector<Data> dataset(100);
    for (int i = 0; i < 100; ++i)
    {
        dataset[i] = {.id = i, .value = i * 10};
    }

    ProcessData(dataset);
    auto var = dataset[42].m_value; // ???
    return 0;
}
```

Watchpoints to the rescue!

- **`watch variable_name`** - Break on write watch *ptr
- **`rwatch variable_name`** - Break on read
- **`awatch variable_name`** - Break on read or write

Watchpoints - example

`day01/06_watchpoints`

VSCode also supports setting watchpoints! (*)

Start Debugging: Launch debug session and pause at a regular breakpoint.

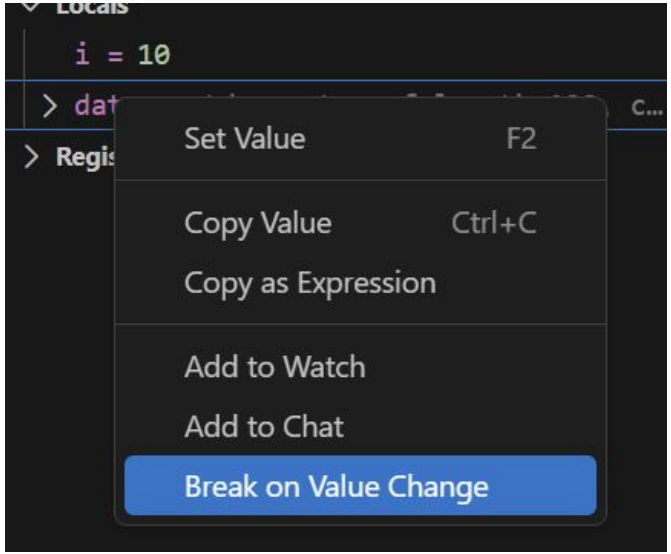
Open the Variables Pane: In the Run and Debug view, locate the **VARIABLES** section.

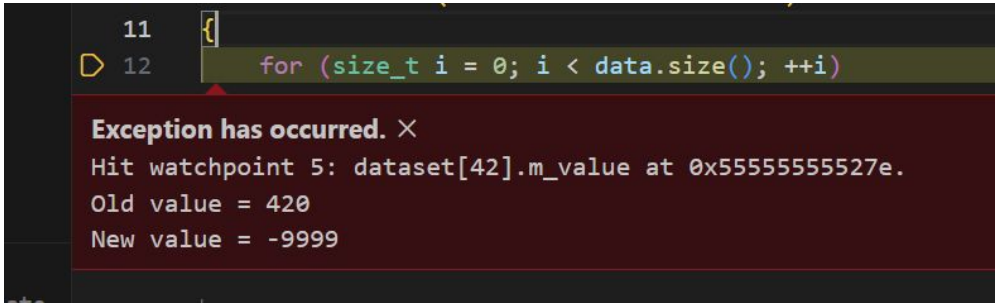
Right-Click the Variable: Right-click the variable, and select **Break on Value Change**

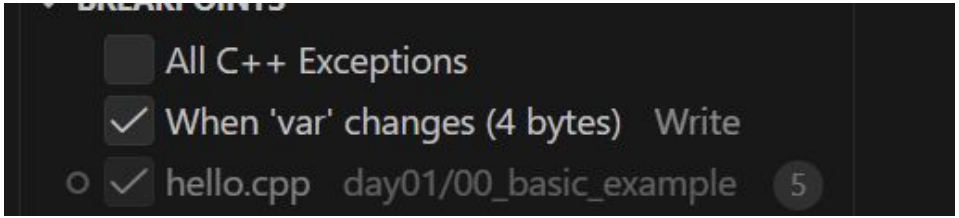
Breakpoints View: Once set, the watchpoint will appear in the **BREAKPOINTS** section

(*) **Doesn’t work on nested variables (array cells)**

How it looks like







Catchpoints

A specialized breakpoint that triggers on **system** or **language events** rather than specific code lines.

- Ideal for debugging "invisible" transitions (e.g., a crash during an exception or a hidden system call).
- Core Event Types
  - **Exceptions:** When thrown or catched
  - **Process Changes:** new process creation (**`fork`**), running a new program (**`exec`**).
  - **System Interactions:** OS-level calls like **open**, **`read`**, or **`write`** (**`syscalls`**)
  - **Library Events:** track load of dynamic libraries

Catchpoint advantages

- **No Source Required:** Stops the program even if the event happens inside a pre-compiled library.
- **Global Scope:** Automatically monitors the entire execution flow without needing a specific function name.
- **Efficiency:** Uses kernel or runtime hooks to pause execution instantly when the event occurs.

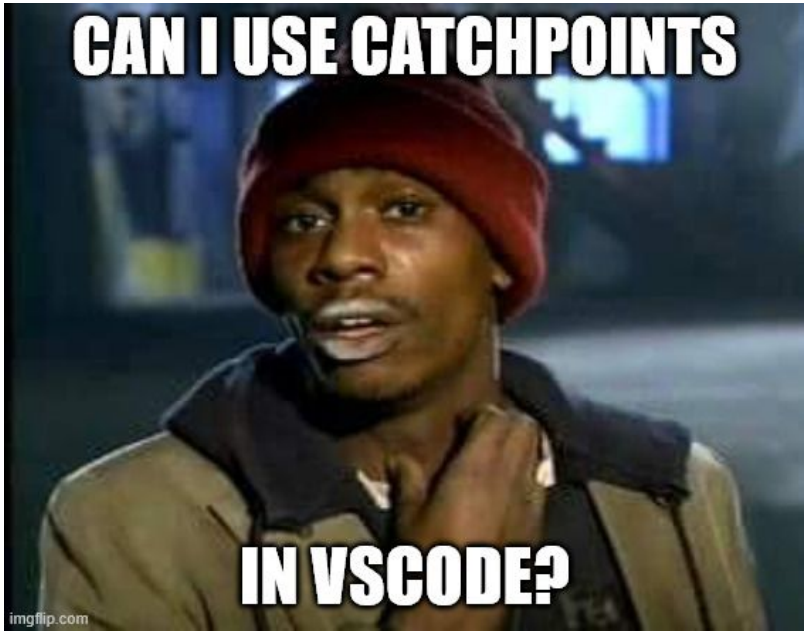
Catchpoints commands

- ~~`info catch`~~ `info break` - Show active try/catch blocks
- `catch throw` - Break when C++ exception is thrown
- `catch catch` - Break when exception is caught
    - `print $_exception` - Displays thrown object (*)
- `catch signal SIGSEGV` - Break on specific signal
- `catch fork` - Break on fork() call
- `catch exec` - Break on exec() call
- `tcatch` - same as catch, but deleted after hit once

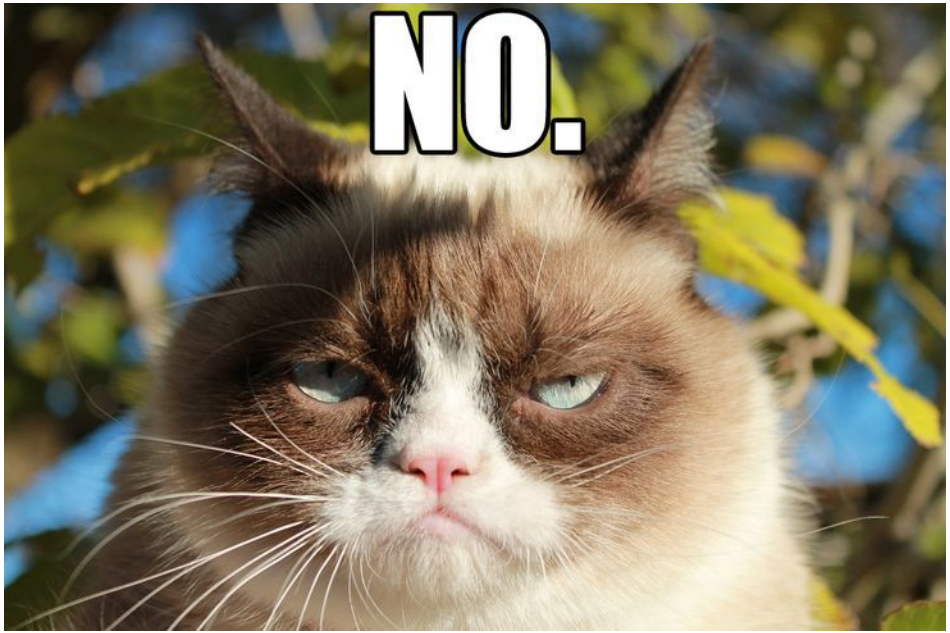
Catchpoints - example

`day01/07_catchpoints`

Catchpoints in VSCode



Catchpoints in VSCode



Exercise


(gdb)
Breakpoint 1 at 0x1234:
file main.c, line 10
Program received signal SIGINT,
Interrupt.
Try 'step' to execute line by line
Use 'print var' to inspect variables
Set breakpoints with 'break function'

Exercise

exercises/day01/ex04/README.md

Summarizing Exercise

Summarizing Exercise (1)

Below is a program that fails a few tests.

Tests reportedly passed in the past, but we are not sure if this is correct. Maybe some tests are faulty and should be amended, maybe some never passed before.

Your task is to review the tests, then debug the code and determine if there is a bug or bugs in the code, if the tests should be amended, or both.

Code: https://godbolt.org/z/T1Tc3j75d

But wait, there is some more info in the next slide.

Summarizing Exercise (2)

A previous version of the code in the source control, seems worse, as it crashed for one of the tests:

https://godbolt.org/z/1hKP9b3n9

It seems that someone was trying to fix the code (creating the newer version in the previous slide), but didn't finish the fix, or was not able to handle all issues.

You may use the old version of the code (link in this slide), or thre newer one (link in previous slide) or both, to pinpoint the issue(s) and fix them.

Summarizing Exercise (3)

Steps:

- Read the tests and the code on your own.
- Do not to look for the bug in a “code review” appraoch, but rather use GDB to debug the code, trying to understand the flow.
- You may decide to debug the old code, or the new code, or to compare them, to gain a better view of the issue, and then debug one of them.
- Before you apply any fix make sure to have a theory of what’s wrong.

Summarizing Exercise - Solution in Next Slides

Don’t peek...

Summarizing Exercise - Solution (1)

There is a small bug in the **old version of the code**: sorting is in ascending order instead of descending order. Also a few tests are faulty.

The bug in the old version of the code is in this line:

```
std::ranges::sort(small_buckets_sorted); // now it is sorted!
```

Should become:

```
std::ranges::sort(small_buckets_sorted, std::greater()); // now it is sorted!
```

Summarizing Exercise - Solution (2)

The new code, instead of just fixing the sort order, tries to bypass the crash (which is related to the wrong sort order) with various poor fixes.

If you focused on the old code you may got better view of the issue, with less noise. But, both approaches should be able to find and fix the bugs.

This is the amended code (with both the bug fix and the amended tests):

https://godbolt.org/z/jPejf5afe

Part 2 Summary

- Debug symbols
- VSCode and GDB
- Advanced breakpoints
  - Conditional
  - Watchpoints
  - Catchpoints

Day 1 Summary

- Basic GDB CLI usage and commands
- Program data display
- Breakpoints
- GDB TUI
- Debug symbols
- VSCode and GDB
- Advanced breakpoints
  - Conditional
  - Watchpoints
  - Catchpoints

What's next?

- Core dump analysis
- Attaching to a running program
- Altering and reverse execution
- Remote debugging
- Debugging various memory issues